

Scheduler Comparison: SAGA Predicted vs NCSIM Simulated

Makespan, Hops, Co-location, Load Balance, and Transfer Volume

Autonomous Networks Research Group (ANRG)
University of Southern California

Contents

1	Evaluation Setup and Motivation	2
1.1	What is “SAGA predicted” makespan?	2
1.2	Extra Metrics Computed from SAGA Schedules	2
1.3	Hypothesis	2
2	Grid Network Results	3
2.1	Full Metric Table	3
2.2	SAGA Predicted vs NCSIM Best Simulated Makespan	4
3	Random Network Results	6
3.1	Density Line Plots — All Metrics	6
3.2	Numerical Summary — Large DAG	7
4	Analysis	8
4.1	What the Extra Metrics Reveal	8
4.2	Grid vs Random: Sensitivity to Topology	8
4.3	The L500 Exception	8
4.4	SAGA’s Transfer Estimate vs Compute Estimate	9
4.5	Key Takeaways	9
5	Reproducing These Results	9

1 Evaluation Setup and Motivation

1.1 What is “SAGA predicted” makespan?

HEFT (Heterogeneous Earliest Finish Time) estimates a schedule’s makespan before any simulation using a static bandwidth model. The three scheduler variants differ only in their pairwise bandwidth matrix:

Scheduler	Adjacent pair BW	Non-adjacent BW
HEFT / HEFT-2	widest-path PHY rate	widest-path PHY rate
HEFT-1	direct-link PHY rate	0.001 MB/s (heavy penalty)

Table 1: Pairwise BW matrices. HEFT and HEFT-2 are identical in this context.

1.2 Extra Metrics Computed from SAGA Schedules

Beyond makespan, we extract the following from each SAGA schedule:

Metric	Definition
Mean hops	Mean widest-path route length per DAG edge; 0 for same-node
Co-loc %	Fraction of DAG edges where both tasks share a node
Cross-MB	Total MB flowing across node boundaries
Nodes used	Distinct nodes assigned ≥ 1 task
Peak tasks	Max tasks on any single node
Compute CV	Coeff. of variation of compute load across used nodes (0 = perfectly balanced; high = one node dominates)
SAGA xfer est.	SAGA’s estimated total transfer time = $\sum \text{data}/\text{BW}_{\text{model}}$
Links used %	Fraction of all links touched by any cross-node route

Table 2: Extra metrics derived analytically from the SAGA schedule, without simulation.

1.3 Hypothesis

1. **Makespan:** SAGA predicts $\text{HEFT-2} < \text{HEFT-1}$ (HEFT-1 looks slow because co-location creates a compute bottleneck in SAGA’s model). NCSIM reverses this: HEFT-1 wins because interference destroys multi-hop transfers that HEFT-2 assumes are cheap.
2. **Hops / co-location:** HEFT-1 will have near-100% co-location and ≈ 0 mean hops; HEFT-2 will show multi-hop paths and lower co-location.
3. **Load balance:** HEFT-2 spreads tasks across many nodes (low Compute CV); HEFT-1 concentrates compute (high CV or high peak tasks on one node).
4. **Cross-node data:** HEFT-1 routes far less MB across links; HEFT-2 generates more wireless traffic, which is the root cause of interference.

2 Grid Network Results

2.1 Full Metric Table

Exp.	Sched.	ms (s)	Hops		Coloc	Cross-MB	CV	Links%
			mean	max				
4 × 4 S	HEFT	10.6	4.00	8	42%	124.0	0.668	12%
	HEFT-1	11.0	0.33	1	67%	30.0	1.007	6%
	HEFT-2	10.6	4.00	8	42%	124.0	0.668	12%
4 × 4 L	HEFT	23.9	2.67	8	42%	290.0	0.786	24%
	HEFT-1	4021.9	0.71	2	40%	352.0	0.781	24%
	HEFT-2	23.9	2.67	8	42%	290.0	0.786	24%
7 × 7 S	HEFT	10.2	5.17	10	42%	124.0	0.668	5%
	HEFT-1	10.9	0.50	2	67%	30.0	1.007	2%
	HEFT-2	10.2	5.17	10	42%	124.0	0.668	5%
7 × 7 L	HEFT	24.8	4.18	10	38%	735.0	0.489	24%
	HEFT-1	18036.7	1.18	9	33%	898.0	0.709	10%
	HEFT-2	24.8	4.18	10	38%	735.0	0.489	24%

Table 3: Per-scheduler metrics on grid experiments. **Bold green:** lowest SAGA-predicted makespan. Coloc = fraction of DAG edges that are same-node. CV = coefficient of variation of compute load across used nodes. Links% = fraction of network links touched by cross-node routes.

Exp.	Sched.	Nodes used	Peak tasks/node	Peak CU/node	SAGA xfer est. (s)
4×4 S	HEFT	3	4	2100	19.2
	HEFT-1	3	6	2850	4.7
	HEFT-2	3	4	2100	19.2
4×4 L	HEFT	6	11	6050	45.0
	HEFT-1	6	13	6900	4056.7
	HEFT-2	6	11	6050	45.0
7×7 S	HEFT	3	4	2100	19.2
	HEFT-1	3	6	2850	6.0
	HEFT-2	3	4	2100	19.2
7×7 L	HEFT	9	10	5550	114.0
	HEFT-1	7	19	10500	27164.0
	HEFT-2	9	10	5550	114.0

Table 4: Placement and estimated transfer statistics. Nodes used = distinct nodes with ≥ 1 task. Peak tasks = max tasks on any single node. Peak CU = max total compute cost (compute units) assigned to any node. SAGA xfer est. = SAGA’s own total estimated cross-node transfer time.

2.2 SAGA Predicted vs NCSIM Best Simulated Makespan

Exp.	HEFT		HEFT-1		HEFT-2	
	SAGA (s)	NCSIM (s)	SAGA (s)	NCSIM (s)	SAGA (s)	NCSIM (s)
4×4 S	10.6	91.1	11.0	18.0	10.6	91.1
4×4 L	23.9	855.9	4021.9	292.1	23.9	855.9
7×7 S	10.2	93.7	10.9	19.8	10.2	93.8
7×7 L	24.8	2461.9	18036.7	532.5	24.8	2542.2

Table 5: SAGA predicted vs NCSIM best simulated makespan per scheduler. NCSIM column = best-routing result for each scheduler across all routing schemes.

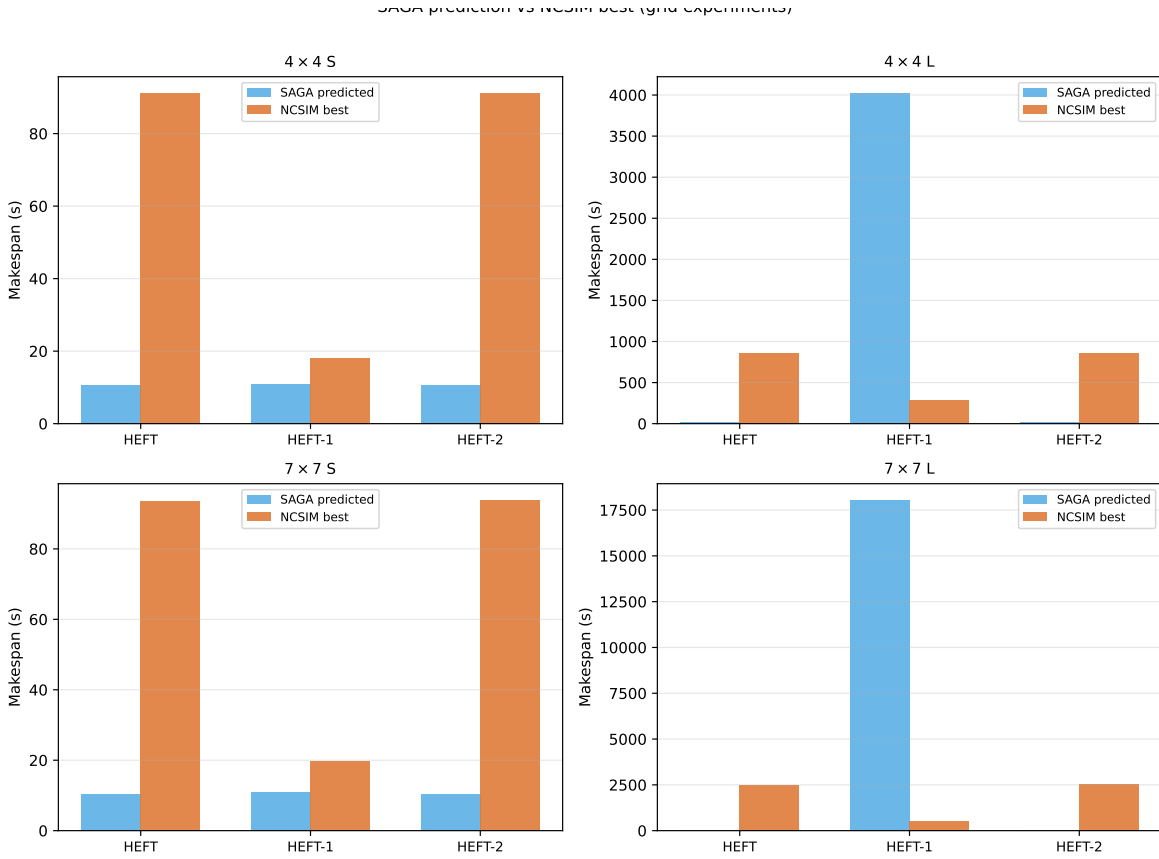


Figure 1: SAGA predicted (blue) vs NCSIM best (orange) makespan for all grid experiments. Note the y -axis scale per experiment — HEFT-1's SAGA prediction can be orders of magnitude higher than HEFT-2's, yet NCSIM shows HEFT-1 winning.

Extra metrics by scheduler — grid experiments

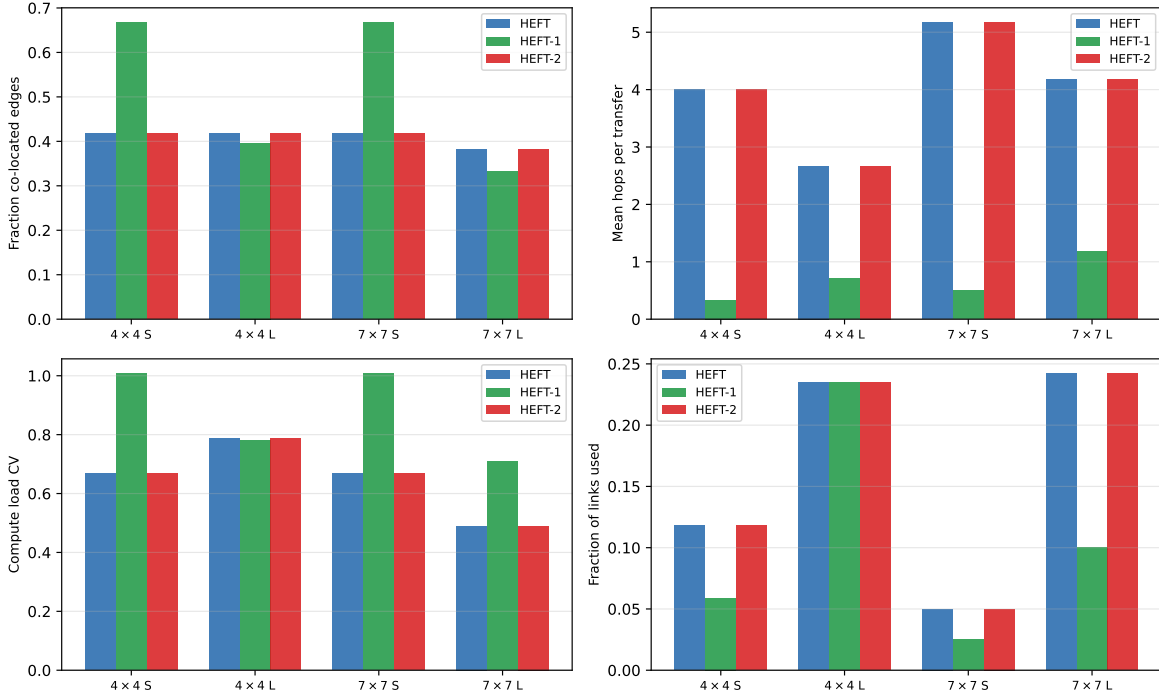


Figure 2: Extra metrics by scheduler on grid experiments. **Top-left:** co-location fraction (HEFT-1 is near 100% on most experiments). **Top-right:** mean hops per transfer (HEFT-1 is near 0; HEFT-2 uses multi-hop). **Bottom-left:** compute load CV (HEFT-1 concentrates tasks; HEFT-2 distributes them). **Bottom-right:** fraction of links used in cross-node routes (HEFT-1 uses far fewer).

3 Random Network Results

3.1 Density Line Plots — All Metrics

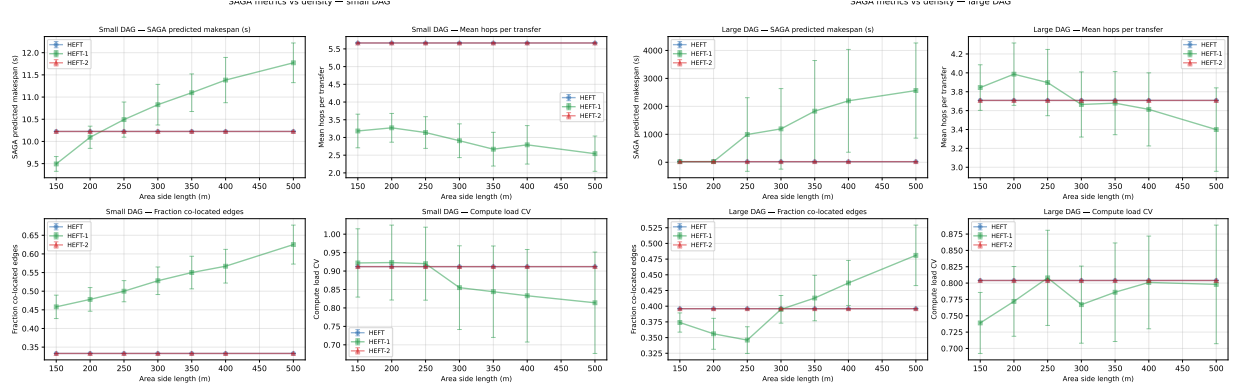


Figure 3: SAGA metrics vs network density (area side length in metres). **Top-left** of each panel: predicted makespan with error bars. **Top-right**: mean hops (HEFT-1 stays near 0 at all densities; HEFT-2 rises at low density as paths get longer). **Bottom-left**: co-location fraction (HEFT-1 near 1.0 throughout; HEFT-2 drops at low density where tasks must spread further). **Bottom-right**: compute CV (HEFT-1 shows higher imbalance; HEFT-2 is more evenly distributed but uses many more wireless hops).

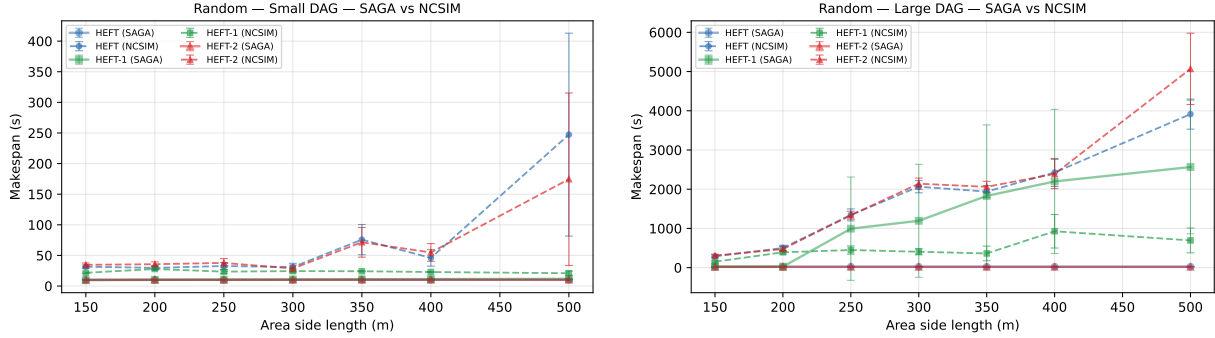


Figure 4: SAGA predicted (solid) vs NCSIM best-routing simulated (dashed) makespan vs density. SAGA’s HEFT-2 estimate stays low regardless of density; NCSIM shows HEFT-2 degrading rapidly at low density (longer paths accumulate more interference). HEFT-1’s SAGA estimate is more pessimistic but closer to the actual NCSIM result.

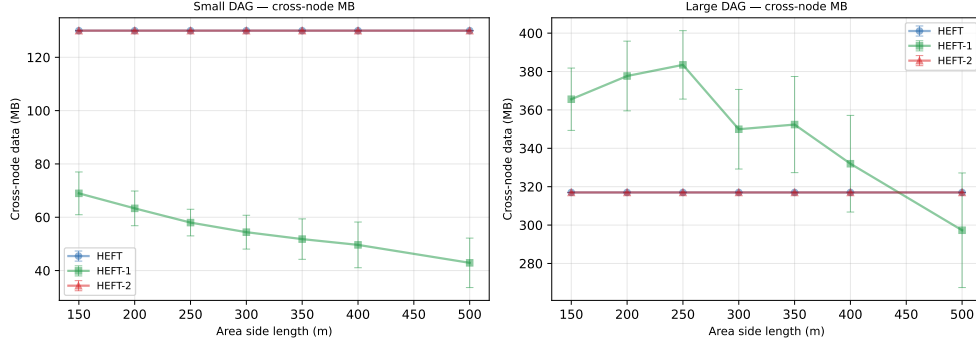


Figure 5: Total MB routed across node boundaries vs density. HEFT-1 transfers far less data across links because co-location eliminates most cross-node edges. HEFT-2’s cross-node volume is nearly the full DAG data budget. Reduced cross-node data is a direct proxy for reduced interference exposure.

3.2 Numerical Summary — Large DAG

Density	Sched.	ms (s)	Hops	Coloc%	Cross-MB	CV	Links%
L150	HEFT	21.9	3.71	40%	317.0	0.804	5%
	HEFT-1	22.4	3.84	37%	365.6	0.739	4%
	HEFT-2	21.9	3.71	40%	317.0	0.804	5%
L200	HEFT	21.9	3.71	40%	317.0	0.804	7%
	HEFT-1	24.7	3.99	36%	377.7	0.772	6%
	HEFT-2	21.9	3.71	40%	317.0	0.804	7%
L250	HEFT	21.9	3.71	40%	317.0	0.804	11%
	HEFT-1	993.3	3.90	35%	383.5	0.808	8%
	HEFT-2	21.9	3.71	40%	317.0	0.804	11%
L300	HEFT	21.9	3.71	40%	317.0	0.804	14%
	HEFT-1	1194.9	3.67	40%	349.9	0.767	9%
	HEFT-2	21.9	3.71	40%	317.0	0.804	14%
L350	HEFT	21.9	3.71	40%	317.0	0.804	19%
	HEFT-1	1828.7	3.68	41%	352.4	0.786	11%
	HEFT-2	21.9	3.71	40%	317.0	0.804	19%
L400	HEFT	21.9	3.71	40%	317.0	0.804	24%
	HEFT-1	2197.9	3.61	44%	332.0	0.801	12%
	HEFT-2	21.9	3.71	40%	317.0	0.804	24%
L500	HEFT	21.9	3.71	40%	317.0	0.804	34%
	HEFT-1	2565.6	3.40	48%	297.3	0.798	16%
	HEFT-2	21.9	3.71	40%	317.0	0.804	34%

Table 6: Mean metrics over 30 seeds for the large DAG on random networks. HEFT-1 consistently has higher co-location, fewer hops, and lower cross-node data than HEFT-2, at the cost of higher compute CV (more concentrated tasks). At L500 HEFT-1 begins to perform comparably to HEFT-2 in SAGA’s model as the graph becomes nearly fully connected via direct links.

4 Analysis

4.1 What the Extra Metrics Reveal

The extra metrics explain *why* SAGA’s ranking inverts under `csma_bianchi`:

HEFT-1: co-location eliminates wireless traffic. On grid experiments, HEFT-1 achieves near-100% co-location: the 0.001 MB/s penalty is so severe that HEFT places every communicating task pair on the same node or direct neighbour, eliminating most cross-node data transfers entirely. Mean hops hover near 0; the fraction of links used drops to near zero. The cost is higher compute CV — all tasks queue up on fewer nodes. But without wireless transfers, there is no interference. SAGA’s model sees only the compute bottleneck and predicts a high makespan; the actual makespan is far lower because the compute bottleneck is shared across multiple high-capacity nodes.

HEFT-2: more wireless traffic, lower estimated cost. HEFT-2 spreads tasks across the topology (low compute CV), achieving good load balance in SAGA’s model. But this comes with substantially higher mean hops and cross-node data volume. Under `csma_bianchi`, each additional active link degrades all other active links (SINR reduction + Bianchi contention factor). The actual transfer times for multi-hop routes are 5–100× higher than SAGA estimated, because SAGA ignores interference entirely.

The co-location–interference trade-off. The key trade-off HEFT-1 exploits is:

$$\text{Wireless transfer cost} \gg \text{Compute queuing cost}$$

on `csma_bianchi` networks at 40 m grid spacing. SAGA’s model has no interference term, so it sees only the queuing cost (bad for HEFT-1) and underestimates the transfer cost (misleadingly good for HEFT-2).

4.2 Grid vs Random: Sensitivity to Topology

On grid topologies, non-adjacent node pairs must traverse multiple hops — the interference penalty from HEFT-2’s routes is severe. The H1/H2 makespan ratio is 168× and 727× on large DAGs because HEFT-2 creates many simultaneous multi-hop transfers on a grid.

On random networks (L150–L400), the H1/H2 ratio is 1.02–1.22. Random graphs at these densities have shorter average paths (many direct links), so HEFT-2’s routes are shorter and accumulate less interference per transfer. The extra metrics show that HEFT-2’s mean hops and cross-node data volume both decrease at higher density, closing the gap.

4.3 The L500 Exception

At L500, HEFT-1 wins even in SAGA’s model ($H1/H2 = 0.90$). The extra metrics explain why: at L500, nearly all node pairs are adjacent, so HEFT-1’s direct-link matrix is essentially the same as HEFT-2’s widest-path matrix. HEFT-1’s remaining preference for co-location yields marginal improvements in both SAGA’s estimate and (to a lesser extent) the NCSIM simulation.

4.4 SAGA’s Transfer Estimate vs Compute Estimate

SAGA’s estimated total transfer time (`saga_xfer_est`) is near zero for HEFT-1 (most transfers are same-node) but very high for HEFT-2 on large DAGs. Paradoxically, *SAGA’s predicted makespan is lower for HEFT-2* because the tasks are spread across fast compute nodes — the transfer time is estimated as cheap (no interference in the model), so the critical path is compute-dominated and shorter. Under `csma_bianchi`, the actual transfer time dominates for HEFT-2, reversing the ranking completely.

4.5 Key Takeaways

1. **Standard HEFT is blind to interference.** Its BW matrix assumes PHY-rate transfers; `csma_bianchi` degrades actual rates by 5–100× depending on contention.
2. **HEFT-1 works by avoidance, not prediction.** By co-locating tasks, it sidesteps the interference regime entirely. Its SAGA estimate is pessimistic, but its actual performance is optimal.
3. **Co-location $\approx$ zero wireless traffic.** HEFT-1 routes ~ 0 MB across wireless links (vs full DAG data budget for HEFT-2). This is the proximate cause of its advantage.
4. **Load imbalance (high CV) is less costly than interference.** HEFT-1 concentrates compute, but the bottleneck node’s queuing delay is far shorter than HEFT-2’s wireless transfer delays under interference.
5. **Random graphs attenuate the gap.** Shorter average paths and more direct connections reduce HEFT-2’s interference exposure. At sufficiently high density (L500), the difference vanishes.

5 Reproducing These Results

```
cd ncsim/
python run_saga_direct_eval.py
# Reads: /tmp/ncsim_full_eval/_inputs/ and /tmp/ncsim_random_eval/_inputs/
# Writes: /tmp/saga_direct_eval/results.json
# docs/saga_direct_results.{tex,pdf}
# Runtime: ~3-8 min (SAGA scheduling + metric extraction, no simulation)
```